

PaperBoost | Construcción con implementación de patrones de diseño y arquitectura

Campo	Información
Departamento	Departamento de Ciencias de la Computación
Carrera	Ingeniería en Software
Asignatura	Análisis y Diseño de Software
NRC	30723
Integrantes	Viscaino Jordy, Lucas Gongora, Klever Jami
Proyecto	PaperBoost
Documento	Construcción con la Implementación de patrones de diseño y arquitectura V2.0

ANTECEDENTES

1. PaperBoost es una aplicación móvil destinada a la administración sencilla de inventario y ventas en pequeños negocios. El usuario principal es el administrador del establecimiento, por lo que el sistema prioriza formularios breves, mensajes comprensibles y operaciones directas.
2. La arquitectura base del proyecto se organiza en tres capas: Presentación, Lógica de Negocio y Datos. Esta estructura separa la interacción con el usuario, las reglas operativas y el almacenamiento de la información.
3. El análisis previo de patrones seleccionó Singleton como patrón principal y limitó su aplicación a SessionManager. El patrón mantiene una sola sesión activa del administrador y se integra dentro de la capa de Lógica de Negocio, sin reemplazar las tres capas ni crear una capa adicional.
4. La versión 2.0 incorpora el patrón Observer para resolver la comunicación en tiempo real entre la capa de Lógica de Negocio y la capa de Presentación. Cuando el stock de un producto cambia como resultado de una venta, las pantallas de Inventario y de Ventas deben actualizarse automáticamente sin que exista un acoplamiento directo entre ellas y el servicio de ventas.
5. Los requisitos funcionales de nivel 0 de PaperBoost abarcan el núcleo de gestión de productos y el control de acceso. El CRUD priorizado está conformado por RQF-007 Registrar producto, RQF-011 Consultar inventario, RQF-013 Editar producto y RQF-023 Dar de baja producto, junto con RQF-008, RQF-012, RQF-014

y RQF-024 como requisitos asociados. El inicio de sesión se incorpora mediante RQF-001 Validar credenciales y RQF-002 Proteger las credenciales ingresadas.

6. La implementación del patrón Observer se aplica exclusivamente a los cambios de stock generados por ventas. El StockChangeNotifier actúa como sujeto Singleton que mantiene una lista de observadores registrados. Cuando StockService realiza una venta, decrementa el stock en el repositorio y, posteriormente, notifica a todos los observadores registrados. Las pantallas InventoryPage y SalesPage se suscriben como observadores concretos y reaccionan recargando sus datos al recibir la notificación.
7. La implementación utiliza almacenamiento administrado por la aplicación. Los repositorios abstraen la forma de guardar y recuperar datos, permitiendo que la Presentación y la Lógica de Negocio se mantengan independientes del mecanismo concreto de almacenamiento.

OBJETIVO GENERAL

Definir la estructura técnica de PaperBoost para implementar los requisitos funcionales de nivel 0, incluyendo el CRUD priorizado de productos, el inicio de sesión y la actualización automática de inventario y alertas ante cambios de stock, mediante una arquitectura de tres capas integrando el patrón Singleton en SessionManager y StockChangeNotifier, y el patrón Observer en el sistema de notificación de cambios de inventario, de forma que los modelos PlantUML sirvan como guía directa para la construcción del sistema.

OBJETIVOS ESPECÍFICOS

- Delimitar las responsabilidades de Presentación, Lógica de Negocio y Datos.
- Modelar los componentes necesarios para validar el acceso, registrar, consultar, editar y dar de baja productos.
- Ubicar SessionManager y StockChangeNotifier como Singleton dentro de la capa de Lógica de Negocio.
- Definir el patrón Observer para la comunicación desacoplada entre el servicio de ventas y las pantallas de inventario y alertas.
- Definir relaciones permitidas y prohibidas entre capas.
- Proporcionar código PlantUML de arquitectura, componentes, clases y secuencias.
- Establecer una matriz de trazabilidad entre requisitos, clases, métodos, validaciones y criterios de aceptación.
- Convertir los modelos en una guía directa para la construcción del sistema durante los sprints.

DESARROLLO

PARTE 1. DELIMITACIÓN DEL PROBLEMA Y REQUISITOS

FUNCIONALES DE NIVEL

0

1.1 Descripción del problema

El núcleo funcional de PaperBoost debe permitir que el administrador acceda mediante credenciales y, una vez autenticado, registre productos, consulte el inventario, modifique información existente y retire productos que ya no se encuentren disponibles. Estas operaciones no deben concentrarse en las pantallas, porque la interfaz terminaría validando acceso, administrando colecciones y modificando estados directamente.

La solución consiste en conservar una comunicación secuencial: la Presentación captura la acción, un controlador coordina el caso de uso, un servicio aplica las reglas y un repositorio administra los registros. La entidad Product representa los datos del dominio. SessionManager se utiliza únicamente para conservar y consultar la sesión activa.

Adicionalmente, cuando se realiza una venta y el stock de uno o varios productos se decrementa, las pantallas de Inventario y de Ventas deben reflejar el nuevo estado sin que exista una llamada directa desde SaleService hacia InventoryPage o SalesPage. El patrón Observer resuelve esta necesidad definiendo una interfaz StockObserver y un sujeto StockChangeNotifier que notifica automáticamente a todos los observadores interesados cada vez que el stock de un producto cambia.

1.2 Requisitos funcionales de nivel 0

ID	Requisito	Operación	Prioridad	Caso de uso
RQF-007	El programa deberá permitir registrar productos manualmente mediante un formulario.	Crear	Alta	CUC-04
RQF-011	El programa deberá mostrar una lista de los productos registrados en el inventario.	Consultar	Alta	CUC-06
RQF-013	El programa deberá permitir modificar la información de un producto registrado.	Actualizar	Alta	CUC-07
RQF-023	El programa deberá permitir	Baja lógica	Alta	CUC-12

ID	Requisito	Operación	Prioridad	Caso de uso
	dar de baja unproducto registrado.			
RQF-001	El programa deberá permitir aladministrador iniciar sesión mediantecorreo electrónico y contraseña.	Autenticar	Alta	CUC-01
RQF-002	El programa deberá proteger lascredenciales ingresadas por eladministrador durante el proceso deinicio de sesión.	Protegercreden ciales	Alta	CUC-01

1.3 Requisitos asociados

ID	Relación con el CRUD	Aporte
RQF-008	Registro	Valida campos obligatorios y evita SKU repetidos.
RQF-012	Consulta	Permite buscar, filtrar y ordenar productos.
RQF-014	Edición	Permite descartar cambios antes de guardar.
RQF-024	Baja	Solicita confirmación y mantiene el producto activo al cancelar.
RQF-001 / RQF-002	Acceso	Permiten activar una sesión única y proteger las operacionesdel administrador.
RQF-002	Inicio de sesión	Oculto la contraseña, evita su exposición y limpialos campos después de intentos incorrectos ocierre de sesión.

PARTE 2. ARQUITECTURA BASE DE TRES CAPAS

2.1 Responsabilidades de las capas

Capa	Componentes principales	Responsabilidad
Presentación	LoginPage, RegisterPage, ProductFormPage, InventoryPage, SalesPage, HomeShell	Capturar datos, ejecutar acciones de interfaz y mostrar resultados. No valida reglas de negocio ni accede directamente a repositorios. InventoryPage y SalesPage actúan como observadores del patrón Observer.
Lógica de Negocio	AuthController, ProductController, SaleController, StockAlertController, AuthService, ProductService, SaleService, StockAlertService, ProductValidator, SaleValidator, StockAlertValidator, OperationResult, SessionManager, StockChangeListener	Coordinar casos de uso, aplicar validaciones, controlar sesión, construir respuestas, decidir cuándo consultar o modificar datos y notificar cambios de stock a los observadores registrados.
Datos	UserRepository, ProductRepository, SaleRepository, StockAlertRepository, User, Product, Sale, SaleItem, Category, StockAlert	Guardar, buscar, actualizar, listar y cambiar el estado de los registros administrados por la aplicación.

2.2 Reglas de comunicación

- Las vistas se comunican únicamente con controladores.
- Los controladores delegan las reglas a servicios.
- Los servicios consultan repositorios y validadores.
- Los repositorios administran entidades y colecciones.
- Las vistas pueden mostrar entidades o DTO de respuesta, pero no deben

modificarlas directamente.

- SessionManager puede ser consultado por servicios protegidos, pero no almacena productos, ventas ni alertas.
- Los repositorios no conocen las vistas ni los controladores.
- StockChangeNotifier notifica a los observadores registrados (InventoryPage, SalesPage) cuando el stock cambia. Los servicios que provocan cambios de stock invocan al notificador, pero no conocen a los observadores concretos.
- Los observadores se suscriben y desuscriben durante el ciclo de vida de la pantalla (initState / dispose).

2.3 Patrones de diseño integrados

Patrón	Ubicación	Responsabilidad
Singleton	SessionManager	Garantizar una única instancia activa de sesión accesible desde AuthService y servicios protegidos.
Singleton	StockChangeNotifier	Garantizar una única instancia del notificador de cambios de stock accesible desde SaleService y los observadores registrados.
Observer	StockObserver (interfaz abstracta), StockChangeNotifier (sujeto), InventoryPage y SalesPage (observadores concretos)	Permitir que las pantallas de inventario y ventas se actualicen automáticamente cuando el stock cambia, sin acoplamiento directo con el servicio de ventas.

Modelo arquitectónico general en PlantUML

```
@startuml
skinparam shadowing false
skinparam componentStyle rectangle
skinparam packageStyle rectangle
skinparam defaultTextAlignment center
left to right direction

title PaperBoost - Arquitectura de Tres Capas\nSingleton + Observer

skinparam package {
  BackgroundColor<<presentacion>> #D6EAF8
  BorderColor<<presentacion>> #2980B9
  BackgroundColor<<logica>> #D5F5E3
```

```
BorderColor<<logica>> #27AE60
BackgroundColor<<datos>> #FCE4D6
BorderColor<<datos>> #C55A11
}
```

actor "Administrador" as Admin

```
package "Capa de Presentación" <<presentacion>> {
  component "LoginPage" as LoginPage
  component "RegisterPage" as RegisterPage
  component "ProductFormPage" as ProductForm
  component "InventoryPage\n<<observer>>" as Inventory
  component "SalesPage\n<<observer>>" as Sales
}
```

```
package "Capa de Lógica de Negocio" <<logica>> {
  component "AuthController" as AuthController
  component "ProductController" as ProductController
  component "SaleController" as SaleController
  component "StockAlertController" as StockAlertC
  component "AuthService" as AuthService
  component "ProductService" as ProductService
  component "SaleService" as SaleService
  component "StockAlertService" as StockAlertS
  component "ProductValidator" as Validator
  component "OperationResult" as Result
  component "SessionManager\n<<Singleton>>" as Session
  component "StockChangeNotifier\n<<Singleton>>" as Notifier
}
```

```
package "Capa de Datos" <<datos>> {
  component "UserRepository" as UserRepo
  component "ProductRepository" as ProductRepo
  component "SaleRepository" as SaleRepo
  component "StockAlertRepository" as AlertRepo
  component "User" as User
  component "Product" as Product
  component "Sale" as Sale
  component "SaleItem" as SaleItem
  component "Category" as Category
  component "StockAlert" as StockAlert
}
```

Admin --> LoginPage
Admin --> ProductForm
Admin --> Inventory
Admin --> Sales

LoginPage --> AuthController : solicita inicio/cierre
AuthController --> AuthService
AuthService --> UserRepo : valida credenciales
AuthService --> Session : activa / cierra sesión
AuthController --> Result

ProductForm --> ProductController : registrar
Inventory --> ProductController : listar / buscar
Sales --> SaleController : registrar venta
Sales --> StockAlertC : consultar alertas
RegisterPage --> AuthController : registrar cuenta

ProductController --> ProductService
ProductService --> Session : verifica acceso
ProductService --> Validator : valida datos / SKU
ProductService --> ProductRepo : guarda / consulta / actualiza
ProductService --> Result
ProductController --> Result

SaleController --> SaleService
SaleService --> ProductRepo : decrementa stock
SaleService --> SaleRepo : registra venta
SaleService --> Notifier : notifica cambios de stock

StockAlertC --> StockAlertS
StockAlertS --> AlertRepo : gestiona alertas
StockAlertS --> ProductRepo : consulta stock actual

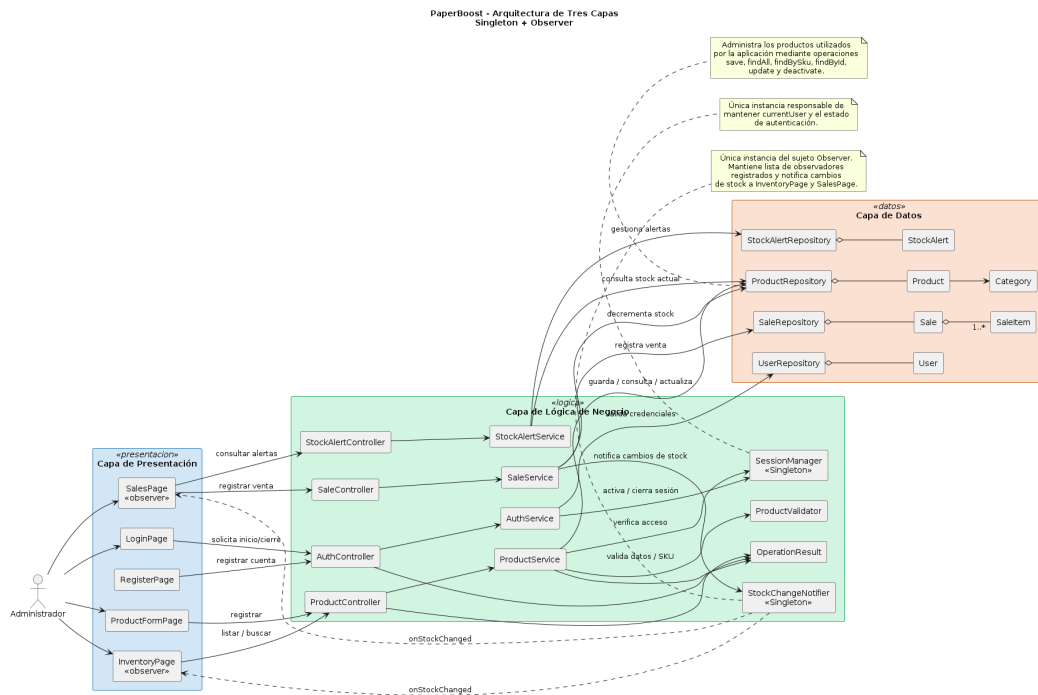
Notifier ..> Inventory : onStockChanged
Notifier ..> Sales : onStockChanged

UserRepo o-- User
ProductRepo o-- Product
SaleRepo o-- Sale
Sale o-- "1..*" SaleItem
AlertRepo o-- StockAlert
Product --> Category

note bottom of Session
Única instancia responsable de
mantener currentUser y el estado
de autenticación.
end note

note bottom of Notifier
Única instancia del sujeto Observer.
Mantiene lista de observadores
registrados y notifica cambios
de stock a InventoryPage y SalesPage.
end note

note bottom of ProductRepo
Administra los productos utilizados
por la aplicación mediante operaciones
save, findAll, findBySku, findById,
update y deactivate.
end note
@enduml



2.4 Definición del patrón Singleton

SessionManager

@startuml

skinparam classAttributelconSize 0

skinparam shadowing false

skinparam packageStyle rectangle

left to right direction

title PaperBoost - Singleton SessionManager

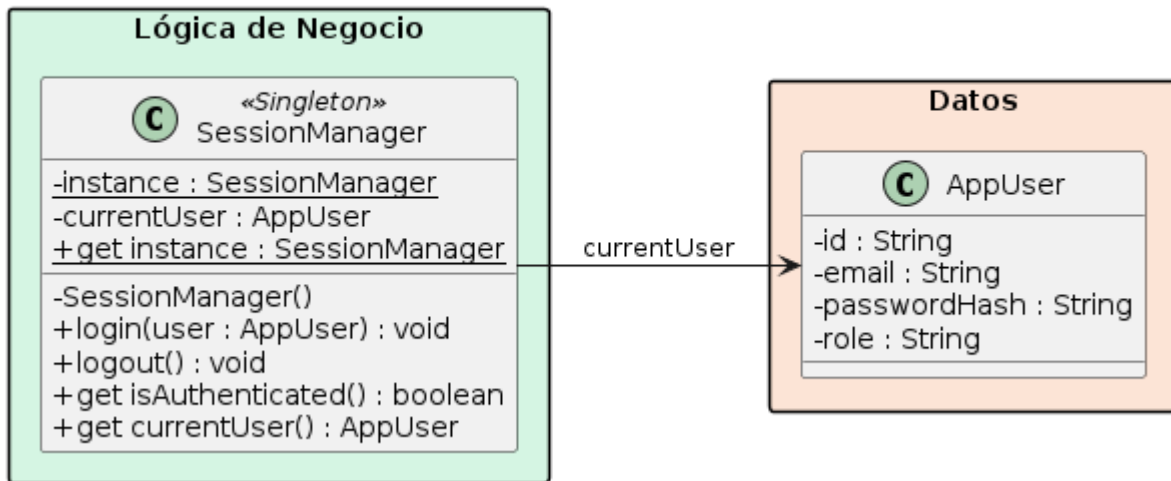
```
package "Lógica de Negocio" #D5F5E3 {
    class SessionManager <<Singleton>> {
        - {static} instance : SessionManager
        - currentUser : AppUser
        - SessionManager()
        + {static} get instance : SessionManager
        + login(user : AppUser) : void
        + logout() : void
        + get isAuthenticated() : boolean
        + get currentUser() : AppUser
    }
}
```

```
package "Datos" #FCE4D6 {
    class AppUser {
        - id : String
        - email : String
        - passwordHash : String
        - role : String
    }
}
```

SessionManager --> AppUser : currentUser

@enduml

PaperBoost - Singleton SessionManager



StockChangeNotifier

```

@startuml
skinparam classAttributeIconSize 0
skinparam shadowing false
skinparam packageStyle rectangle
left to right direction

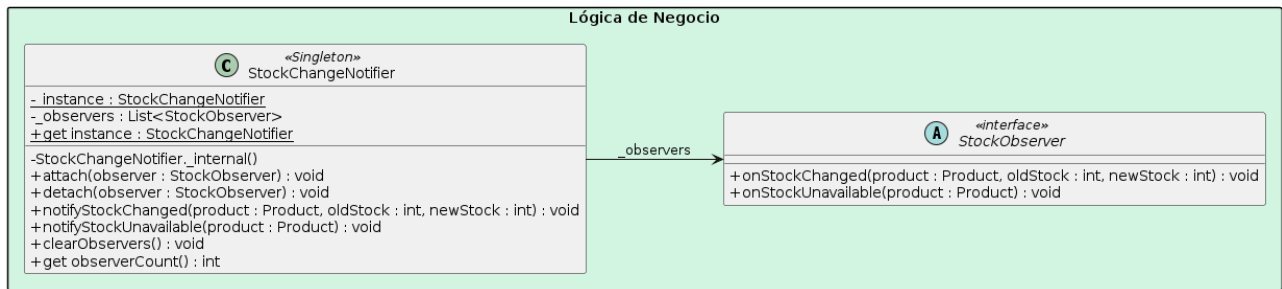
title PaperBoost - Singleton StockChangeNotifier

package "Lógica de Negocio" #D5F5E3 {
    class StockChangeNotifier <<Singleton>> {
        - {static} _instance : StockChangeNotifier
        - _observers : List<StockObserver>
        - StockChangeNotifier._internal()
        + {static} get instance : StockChangeNotifier
        + attach(observer : StockObserver) : void
        + detach(observer : StockObserver) : void
        + notifyStockChanged(product : Product, oldStock : int, newStock : int) : void
        + notifyStockUnavailable(product : Product) : void
        + clearObservers() : void
        + get observerCount() : int
    }

    abstract class StockObserver <<interface>> {
        + onStockChanged(product : Product, oldStock : int, newStock : int) : void
        + onStockUnavailable(product : Product) : void
    }
}

StockChangeNotifier --> StockObserver : _observers
@enduml

```



PARTE 3. PATRÓN OBSERVER - NOTIFICACIÓN DE CAMBIOS DE STOCK

3.1 Definición y propósito

El patrón Observer define una dependencia uno a muchos entre un objeto llamado sujeto y una lista de objetos llamados observadores, de manera que cuando el sujeto cambia de estado, todos sus observadores son notificados y actualizados automáticamente.

En PaperBoost, este patrón se utiliza para resolver el problema de actualización automática de las pantallas de Inventario y de Ventas cuando el stock de un producto cambia como resultado de una venta registrada.

3.2 Componentes del patrón

Rol	Clase	Ubicación	Responsabilidad
Observador abstracto	StockObserver	lib/logic/observers/stock_observer.dart	Definir la interfaz que todos los observadores deben implementar: onStockChanged y onStockUnavailable.
Sujeto (notificador)	StockChangeNotifier	lib/logic/observers/stock_change_notifier.dart	Administrar la lista de observadores registrados y disparar las notificaciones cuando el stock cambia. Es un Singleton.
Disparador del evento	SaleService	lib/logic/services/sale_service.dart	Al crear o cancelar una venta, decrementa o restaura el stock en el repositorio y notifica al StockChangeNotifier.

Rol	Clase	Ubicación	Responsabilidad
Observador concreto 1	InventoryPage	lib/presentation/pages/inventory_page.dart	Se suscribe como observer en initState, recarga la lista de productos al recibir onStockChanged y se desuscribe en dispose.
Observador concreto 2	SalesPage	lib/presentation/pages/sales_page.dart	Se suscribe como observer en initState, recarga el conteo de alertas críticas al recibir onStockChanged y se desuscribe en dispose.

3.3 Interfaz del Observador abstracto

```

@startuml
skinparam classAttributeFontSize 0
skinparam shadowing false

title PaperBoost - Interfaz StockObserver

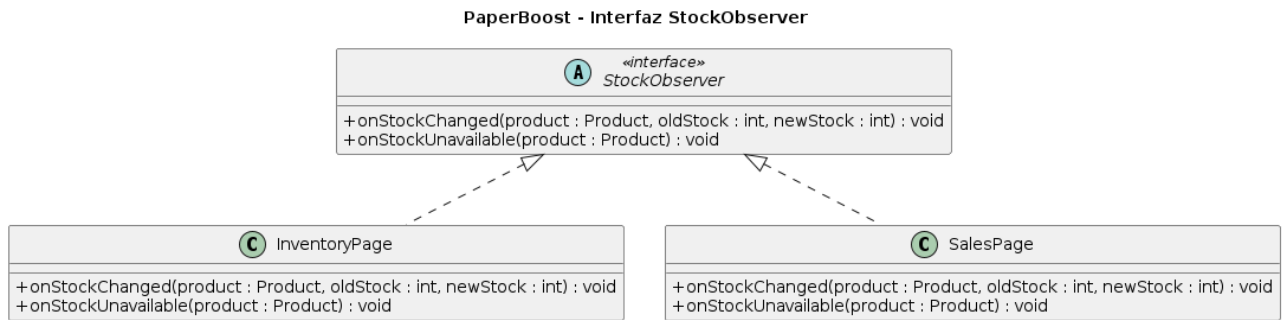
abstract class StockObserver <<interface>> {
+ onStockChanged(product : Product, oldStock : int, newStock : int) : void
+ onStockUnavailable(product : Product) : void
}

class InventoryPage {
+ onStockChanged(product : Product, oldStock : int, newStock : int) : void
+ onStockUnavailable(product : Product) : void
}

class SalesPage {
+ onStockChanged(product : Product, oldStock : int, newStock : int) : void
+ onStockUnavailable(product : Product) : void
}

StockObserver <|.. InventoryPage
StockObserver <|.. SalesPage
@enduml

```



PARTE 4. RESUMEN DE INTEGRACIÓN DE PATRONES

Patrón	Clase principal	Capa	Función en el sistema
Singleton	SessionManager	Lógica de Negocio	Garantiza una única instancia de sesión activa accesible desde AuthService y servicios protegidos.
Singleton	StockChangeNotifie r	Lógica de Negocio	Garantiza una única instancia del notificador de cambios de stock accesible desde SaleService y los observadores.
Observer	StockObserver (interfaz)	Lógica de Negocio	Define el contrato que los observadores deben implementar: onStockChanged y onStockUnavailable.
Observer	StockChangeNotifie r (sujeto)	Lógica de Negocio	Administra la lista de observadores y dispara notificaciones cuando el stock de un producto cambia.
Observer	InventoryPage (observador)	Presentación	Se suscribe como observer, recarga la lista de productos al recibir notificaciones de cambio de stock.
Observer	SalesPage (observador)	Presentación	Se suscribe como observer, recarga el

Patrón	Clase principal	Capa	Función en el sistema
			conteo de alertas críticas al recibir notificaciones de cambio de stock.

REFERENCIAS

- G1_30723_U2T123 - Arquitectura de tres capas e integración de patrones de diseño.
- Gamma, E., Helm, R., Johnson, R. y Vlissides, J. Design Patterns: Elements of Reusable Object-Oriented Software.
- Sommerville, I. Ingeniería de Software.

Ingeniería en Software - Análisis y Diseño de Software - NRC 30723